Constructive and Destructive Training Regimes: A Framework for Interpreting SGD Stability

Thiago Mattar, Cristiano L. Castro, Frederico Coelho, Yan G. Ferreira, Antonio P. Braga

***Abstract*—Stability in stochastic gradient descent (SGD) is typically analyzed using pointwise criteria such as local curvature and gradient-noise statistics. Although these perspectives are useful for characterizing whether an individual update is locally stable, they are less informative about how sequential updates interact with each other. This leaves an important part of optimization unexplained: why some training choices lead to smooth and robust progress, while others produce oscillation, cancellation, or drift. In this paper, we introduce a signal–interference framework for stochastic optimization that makes these trajectory interactions explicit. Through a finite-time decomposition of parameter growth into gradient energy, intra-batch alignment, and cross-step interference, we show that training dynamics depend not only on the strength of individual gradients, but also on how gradients from different mini-batches interact across time, shaping both parameter growth and loss reduction. Using this framework, we study how momentum and weight decay improve the performance of plain SGD, providing additional insight into the mechanisms underlying these optimization strategies. In particular, our results show that plain SGD often operates in destructive regimes, whereas momentum promotes constructive accumulation of updates and weight decay improves robustness by introducing a recency bias. Additionally, we show that batch representativeness is a major driver of trajectory coherence, helping to prevent destructive interference between successive updates. Together, these results help clarify the factors shaping SGD stability beyond local analysis, opening a broader research agenda for interpreting training mechanisms, improving online diagnostics, and designing interference-aware algorithms.**

***Index Terms*—Stochastic Gradient Descent, Training Stability, Gradient Interference, Momentum, Weight Decay**

I. Introduction

Neural scaling laws have ushered in new paradigms for interpreting deep learning dynamics. By establishing that performance gains follow predictable behavior relative to model size, dataset scale, and compute, stochastic optimization methods have been pushed toward increasingly extended training horizons while operating under tight efficiency constraints [1]–[4]. In this regime, optimization stability becomes an important practical requirement, since unstable training may result on slow convergence and high sensitivity to hyperparameter choices. In practice, stability is often maintained through a set of heuristics, such as momentum, batch-size scaling, normalization, and learning-rate schedules [5]–[9]. Yet, despite their effectiveness, it is not fully understood why these mechanisms work, and under what conditions they fail.

Training becomes unstable when updates are too large relative to sharp directions of the loss landscape, so that they overshoot rather than converge towards a minimum [10], [11]. Stochastic extensions, such as the break-even point framework, argue that instability depends jointly on sharpness and curvature variance, particularly during an early phase in which learning rate and batch size strongly shape further trajectory [12]. In contrast, however, the Edge-of-Stability literature has shown that deep networks often continue to make progress near, or even beyond, the nominal sharpness threshold, suggesting that curvature alone is not a sufficient explanation [13]–[15]. Such works provide an important geometric interpretation of training dynamics, but leave a central gap: they are fundamentally local. Current approaches describe potential gain at a point, but do not directly explain how the update produced by one mini-batch interacts with the next—whether successive steps reinforce previous progress, decorrelate into noise, or partially overwrite one another. As a result, two training runs may exhibit similar statistics while following very different optimization paths, with one remaining smooth and coherent and the other becoming unstable. Moreover, the metrics that underlie these analysis, such as leading Hessian eigenvalues or gradient-noise covariance, are impractical to estimate online at the scale of modern training.

To narrow this gap, we focus on the stability of multiple steps of the optimization trajectory. We argue that stable training is not defined only by reaching a low-loss solution, but also by whether parameter updates accumulate smoothly, coherently, and incrementally over time, without excessive oscillation or abrupt spikes. This shifts the central question from whether a single step is locally stable to what makes a full SGD trajectory stable. Our central claim is that stochastic learning involves an inherent trade-off between incorporating new information (signal) and overwriting previously learned structure (interference). We argue that this trade-off is not specific to catastrophic forgetting in non-i.i.d. or continual learning settings [16]; it arises even under standard i.i.d. sampling and is controlled by the successive interactions among gradients from different mini-batches.

The intuition is that, when a network already stores many concepts in its weights, back-propagating the error to those weights using a new mini-batch necessarily modifies pre-

Thiago Mattar, and Yan G. Ferreira are with the Graduate Program in Electrical Engineering, Universidade Federal de Minas Gerais, Belo Horizonte 31270-901, Brazil (e-mail: thiagocmattar@gmail.com; yanvgferreira@gmail.com).

Cristiano L. Castro is with the Graduate Program in Electrical Engineering and the Department of Electrical Engineering, Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, Minas Gerais 31270-901, Brazil (e-mail: crislcastro@gmail.com).

Frederico Coelho, and Antonio P. Braga are with the Graduate Program in Electrical Engineering and the Department of Electronic Engineering, Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, Minas Gerais 35931-008, Brazil (e-mail: fredgfc@ufmg.br; apbraga@ufmg.br).

viously learned structure. If the new gradients align with past updates, the change reinforces existing representations; if they do not, it partially overwrites them. When gradients interfere destructively, successive updates partially undo one another, leading to oscillatory parameter trajectories, a reduced effective step size, and slower convergence.

We make this clearer through a finite-time decomposition of parameter growth under mini-batch SGD, isolating initialization effects, gradient signal, and cross-step interference. This decomposition also provides a mechanistic lens for optimization heuristics such as momentum and weight decay, foundational elements of modern training methods [17]–[19]. Through our proposed framework, we can show that plain SGD often operates in a destructive-interference regime, momentum shifts training toward constructive accumulation, and weight decay introduces a short term memory bias that further improves robustness across learning rates. To make the signal–interference framework practical, we also derive an approximation based on the cosine similarity between consecutive mini-batch gradients, (ρ_t), which can be computed in an online setting.

In summary, our main contributions are:

- Description of the signal–interference trade-off in stochastic optimization and its implications for SGD trajectory stability and loss reduction;
- Presentation of a practical tool that enables online diagnostic of trajectory coherence through the consecutive-gradient cosine similarity ρ_t;
- Presentation of empirical evidence that final performance is strongly linked to the signal–interference trade-off, and that momentum and weight decay systematically shift cross-step interactions toward faster and more stable regimes;
- Presentation of controlled experiments showing that batch representativeness is a primary driver of cross-step coherence, where disrupting it collapses ρ_t and induces drift-like dynamics even at large batch sizes.

The remainder of the paper is organized as follows. Section 2 reviews the main existing frameworks for stability analysis. Section 3 introduces the signal–interference trade-off, together with the tractable diagnostic based on consecutive-gradient cosine similarity. Section 4 studies stability control mechanisms—plain SGD, momentum, and weight decay—under the lens of the proposed trade-off and supports the analysis with ablations over learning rates and batch sizes, as well as experiments isolating the role of batch representativeness. Section 5 concludes and discusses interference-aware directions for future algorithm design.

II. Stability Analysis of SGD

Intuitively, stable optimization occurs when updates reinforce each other and gradually shift parameters toward a point of minimum loss. Instability therefore arises when updates overshoot, oscillate, or partially undo the progress made by previous steps [10], [11]. Understanding this behavior requires examining the mechanisms that govern how updates evolve from one iteration to the next. Two factors have traditionally played a central role in explaining these dynamics: the *local curvature* of the loss landscape and the *stochasticity* introduced by mini-batch sampling.

A. Local Curvature of the Loss Function and Stability

Training can be interpreted as a discrete-time dynamical system in which the local curvature of the loss function determines the stability of the updates [10], [11]. The curvature indicates how sensitive the loss is to parameter changes in different directions: when curvature is small, updates tend to produce smooth descent toward a minimum, whereas in directions of high curvature the same step can overshoot the minimum and induce oscillatory behavior [10], [11], [20]. For this reason, classical analyses of gradient descent relate stability to the spectrum of the Hessian of the loss function, whose eigenvalues quantify directional curvature and indicate how sensitive optimization is to the choice of learning rate [10], [11].

Under this view, mini-batch SGD can be written as the time-varying map

$$\theta_{t+1} = \Phi_t(\theta_t) := \theta_t - \eta\, G_{B_t}(\theta_t), \tag{1}$$

where G_{B_t} is the stochastic gradient associated with the batch B_t. Local trajectory stability is governed by the Jacobian of this update map,

$$J_{B_t} \approx I - \eta\, H_{B_t}(\theta_t), \tag{2}$$

in which $H_{B_t}(\theta_t)$ is the mini-batch Hessian. In full-batch gradient descent, H_{B_t} becomes $H_t = \nabla^2 \mathcal{L}(\theta_t)$ and $J_t = I - \eta H_t$.

Then, consider a small perturbation e_t around the parameters at the current step t. Under the local linearization of the update map, this perturbation evolves as

$$e_{t+1} \approx J_t e_t \approx (I - \eta H_t) e_t,$$

where here $H_t = \nabla^2 \mathcal{L}(\theta_t)$ denotes the Hessian of the full loss. For mini-batch SGD, one may analogously consider the mini-batch Hessian H_{B_t}, but the classical stability condition below is exact in the deterministic full-batch. In a locally quadratic regime near a minimum, the Hessian is approximately constant and admits eigenpairs (λ_k, v_k). Decomposing the perturbation as $e_t = \sum_k \delta_t^{(k)} v_k$, each eigendirection evolves independently according to

$$\delta_{t+1}^{(k)} = (1 - \eta\lambda_k)\, \delta_t^{(k)}.$$

Thus, the factor $1 - \eta\lambda_k$ determines whether perturbations along direction v_k decay or grow. A classical necessary and sufficient condition for linear stability is

$$\max_k |1 - \eta\lambda_k| < 1 \;\Leftrightarrow\; \eta\, \lambda_{\max}(H_t) < 2, \tag{3}$$

which means that all perturbation components contract over time [10], [11]. In other words, if $0 < \eta\lambda_k < 1$, the perturbation decays monotonically along v_k; if $1 < \eta\lambda_k < 2$, it alternates sign while shrinking, producing oscillations around the minimum; and if $\eta\lambda_k > 2$, it alternates sign while growing, leading to divergence. In over-parameterized networks, however, the Hessian spectrum is typically highly anisotropic, with a small number of outlier directions carrying most of

the sharpness, while the bulk remains near zero [21], [22]. Although this suggests monitoring $\lambda_{\max}(H_t)$ as a stability diagnostic, estimating leading Hessian eigenvalues online in large neural networks is computationally expensive, and mini-batch estimates provide only noisy, pointwise proxies of the local geometry.

B. Mini-Batch Stochasticity

A complementary line of work studies SGD through the lens of stochastic optimization, interpreting the mini-batch gradient as a noisy approximation of the full gradient computed over the entire dataset [23]–[26]. In this sense, each SGD step combines a deterministic descent component with a zero-mean random fluctuation induced by sampling, which covariance provides information about how noise is distributed across parameter-space directions and how it may affect trajectory stability. Formally, we write the mini-batch gradient as

$$\begin{aligned} G_{B_t}(\theta_t) &= \nabla\mathcal{L}(\theta_t) + \xi_t, \\ \mathbb{E}[\xi_t \mid \theta_t] &= 0, \\ \mathrm{Cov}(\xi_t \mid \theta_t) &\approx \frac{1}{m}\,\Sigma(\theta_t), \end{aligned} \tag{4}$$

where ξ_t is the sampling noise and $\Sigma(\theta_t)$ denotes the covariance matrix of per-sample gradients around the full gradient. Writing $g_i^{(t)} := \nabla\ell(\theta_t; x_i, y_i)$ and the full gradient vector as $g^{(t)} := \frac{1}{N}\sum_{i=1}^{N} g_i^{(t)} = \nabla\mathcal{L}(\theta_t)$, the covariance matrix is

$$\Sigma(\theta_t) := \frac{1}{N}\sum_{i=1}^{N}\big(g_i^{(t)} - g^{(t)}\big)\big(g_i^{(t)} - g^{(t)}\big)^{\top}, \tag{5}$$

and the scalar dispersion of per-sample gradients around the full gradient is given by the trace of $\Sigma(\theta_t)$

$$\mathrm{tr}(\Sigma(\theta_t)) = \frac{1}{N}\sum_{i=1}^{N}\|g_i^{(t)} - g^{(t)}\|^2.$$

This makes clear why $\Sigma(\theta_t)$ is generally not a viable online diagnostic: computing it requires access to the full gradient $g^{(t)}$ and all per-sample gradients $\{g_i^{(t)}\}_{i=1}^{N}$ at each training step, which is impractical and incurs at least a full-pass cost per step.

C. Curvature–Noise Interaction

The *break-even point* framework of Jastrzebski et al. [12] is an attempt to extend the classical curvature-based stability condition to SGD. The central idea is to analyze stability along the sharpest direction of the loss landscape while explicitly accounting for mini-batch stochasticity. More precisely, the framework proposes a one-dimensional necessary-and-sufficient condition for SGD stability at iteration t,

$$(1 - \eta\lambda_{\max}(H_t))^2 + s_t^2\frac{\eta^2(N-m)}{m(N-1)} \;\le\; 1, \tag{6}$$

where $\lambda_{\max}(H_t)$ is the top Hessian eigenvalue, N is the dataset size, m is the batch size, and s_t^2 denotes the variance across training examples of the curvature measured along the leading Hessian direction. Large s_t^2 means different mini-batches see substantially different local curvature, while small s_t^2 means they see similar curvature. In this sense, Eq. (6) can be read as a stochastic extension of the deterministic stability condition in Eq. (3), so that the first term captures curvature-induced amplification, while the second captures the destabilizing effect of mini-batch stochasticity.

Under this definition, the break-even point is defined as the first iteration at which the left-hand side of Eq. (6) reaches 1. Intuitively, it marks the end of an early phase during which the choice of learning rate and batch size can strongly shape the later optimization trajectory. In this sense, a central conjecture of this framework is that, prior to this point, hyperparameters largely determine the maximum curvature and stochasticity levels encountered later in training, thereby setting the regime in which the subsequent dynamics unfold.

Although the local analysis simplifies both the interpretation and the mathematical treatment of the stability problem, it is subject to well-known limitations. For instance, the Edge-of-Stability (EoS) phenomenon challenges the local-quadratic picture even in deterministic gradient descent (GD). Across architectures and tasks, full-batch GD often exhibits progressive sharpening, with the largest eigenvalue of the Hessian, $\lambda_{\max}(H_t)$, rising until it hovers near (and sometimes above) the nominal threshold $2/\eta$ (Eq. 3), while optimization continues to make progress despite short-term non-monotonicity [13]–[15]. Thus, successful training can coexist with local linear instability for extended periods, indicating that sharpness alone does not fully explain and determine trajectory-level behavior.

Together, these results clarify both the value and the limitation of curvature–noise interaction criteria. Quantities such as the leading Hessian eigenvalue $\lambda_{\max}(H_t)$, the gradient-noise covariance $\Sigma(\theta_t)$, and related one-step stability conditions provide a pointwise description of local amplification potential, often along a dominant direction. In practice, however, these quantities are difficult to estimate online during training, especially in large neural networks. More fundamentally, even when they are available, they fail to describe the real training dynamics, as shown by the EoS phenomenon. It could be argued that this is mainly due to the strictly local validity of the discussed frameworks, which makes it difficult to extrapolate the analyses to the full optimization trajectory. A more comprehensive analysis should explain, for instance, how sequential gradient steps interact to amplify or cancel each other's effects. That interaction can be decisive for trajectory stability: two runs may exhibit similar sharpness and similar noise levels, yet follow very different optimization paths because successive updates may either reinforce one another or partially overwrite previous progress. Therefore, we argue that stability is fundamentally a property of the *training trajectory*: it reflects how successive parameter updates interact and accumulate over time.

III. The Signal-Interference Trade-off

To account for the cumulative effects of the training trajectory in the stability analysis of SGD, we propose a decomposition of the parameter vector magnitude into interpretable

terms that quantify both the energy of the updates and the alignment between gradients, measured both within individual batches and across successive batches.

In this sense, consider a neural network $y = f(\mathbf{x}, \theta)$ with parameters $\theta \in \mathbb{R}^d$. Let $\mathcal{L}(\theta)$ be the loss function defined over the training dataset $D = \{(x_i, y_i)\}_{i=1}^N$, where N is the total number of samples. The standard gradient descent update rule is

$$\theta_{t+1} = \theta_t - \eta \, \nabla \mathcal{L}(\theta_t),$$

where $\eta > 0$ is the learning rate, and $\nabla \mathcal{L}(\theta_t)$ is the gradient of the loss with respect to θ at iteration t. Let $B_t \subset D$ be a mini-batch of size $m = |B_t|$ at each iteration t. The mini-batch gradient is defined as

$$G_{B_t}(\theta_t) = \frac{1}{m} \sum_{i=1}^{m} g_i^{(t)},$$

where $g_i^{(t)} = \nabla \ell(\theta_t; x_i, y_i)$ is the gradient of the loss for the i-th sample in batch B_t at iteration t. The parameter update rule using the mini-batch gradient becomes

$$\theta_{t+1} = \theta_t - \eta \, G_{B_t}.$$

If the training dataset is uniformly distributed across mini-batches with m samples, then one epoch consists of $\frac{N}{m}$ iterations, covering the entire dataset once. Training for k epochs involves a total of $T = k \times \frac{N}{m}$ iterations. After T iterations, the final parameter vector θ_T is given by

$$\theta_T = \theta_0 - \eta \sum_{t=0}^{T-1} G_{B_t}. \tag{7}$$

where θ_0 represents the parameters on initialization. Equation 7 shows that the final parameter vector is obtained by accumulating gradient updates along the optimization trajectory. In deep learning, the magnitude and growth of the weight vector are closely related to the convergence behavior of the model [27], [28]. This motivates analyzing the evolution of the parameter norm during training. Thus, taking the squared norm of θ_T, we can derive the following expression

$$\begin{aligned} \|\theta_T\|^2 &= \Big\| \theta_0 - \eta \sum_{t=0}^{T-1} G_{B_t} \Big\|^2 \\ &= \|\theta_0\|^2 - 2\eta \sum_{t=0}^{T-1} \theta_0^\top G_{B_t} + \eta^2 \Big\| \sum_{t=0}^{T-1} G_{B_t} \Big\|^2. \end{aligned} \tag{8}$$

This gives us three terms: (i) the norm of the parameters on initialization, (ii) the cross-alignment term between weight initialization and gradient trajectory, and (iii) the gradient trajectory. Let's call (ii) the *initialization-trajectory alignment*

$$C := -2\eta \sum_{t=0}^{T-1} \theta_0^\top G_{B_t}. \tag{9}$$

C captures the alignment between the network initial parameters θ_0 and the cumulative gradient trajectory. The last term $\eta^2 \Big\| \sum_{t=0}^{T-1} G_{B_t} \Big\|^2$ gives the norm contribution of the gradient over all iterations (trajectory). As $G_{B_t} = \frac{1}{m} \sum_{i=1}^{m} g_i^{(t)}$, this term can be expanded as follows

$$\begin{aligned} \Big\| \sum_{t=0}^{T-1} \sum_{i=1}^{m} g_i^{(t)} \Big\|^2 &= \sum_{t=0}^{T-1} \sum_{i=1}^{m} \sum_{u=0}^{T-1} \sum_{j=1}^{m} g_i^{(t)\top} g_j^{(u)} \\ &= \underbrace{\sum_{t=0}^{T-1} \sum_{i=1}^{m} \|g_i^{(t)}\|^2}_{S_1} \\ &\quad + \underbrace{\sum_{t=0}^{T-1} \sum_{i=1}^{m} \sum_{\substack{j=1 \\ j \neq i}}^{m} g_i^{(t)\top} g_j^{(t)}}_{S_2} \\ &\quad + \underbrace{\sum_{t=0}^{T-1} \sum_{u=0}^{t-1} \sum_{i=1}^{m} \sum_{j=1}^{m} 2 \, g_i^{(t)\top} g_j^{(u)}}_{S_3}. \end{aligned} \tag{10}$$

The summation above lists every inner product between two sample–level gradients $g_i^{(t)}$ and $g_j^{(u)}$ collected during training. We partition those index pairs into three sets:

- S_1 **(per–sample gradient energy)** gathers the diagonal terms where the two indices coincide ($t = u,\ i = j$). It therefore measures the total magnitude of every individual sample gradient over all iterations.
- S_2 **(intra-batch alignment)** retains the pairs drawn *within the same iteration* but from *different* samples ($t = u,\ i \neq j$). It therefore measures how gradients from different examples in the same mini-batch interact. It quantifies the degree of directional alignment between these gradients, reflecting the semantic similarity between samples. For instance, when a batch contains images with similar content or class labels, their corresponding gradients tend to align, resulting in a positive S_2. This constructive alignment reinforces learning by increasing the effective step size. Conversely, a negative S_2 indicates conflicting gradients—often arising from uncorrelated or contradictory samples—leading to partial signal cancellation within the batch.
- S_3 **(cross-step interference)** collects all gradient inner products across different training iterations ($t \neq u$). It measures how gradients from earlier batches align or interfere with gradients from later ones. A large positive S_3 indicates that successive mini-batches produce correlated gradient directions, leading to a coherent trajectory in parameter space and more efficient accumulation of learning signal. This should occur when consecutive batches contain similar samples whose gradients reinforce each other. In contrast, when the content of consecutive batches is uncorrelated, gradient directions tend to decorrelate, causing S_3 to shrink. At the other extreme, $S_3 < 0$ implies destructive interference, in which updates from one batch partially undo progress made by earlier batches, leading to oscillatory or unstable learning dynamics.

Together, we obtain the final norm decomposition

$$\boxed{\|\theta_T\|^2 \;=\; \|\theta_0\|^2 \;+\; C \;+\; \frac{\eta^2}{m^2}\big(S_1 + S_2 + S_3\big).} \tag{11}$$

Equation 11 explains the magnitude of the parameter vector at any training step, as the result of three conceptually distinct terms: (i) the baseline magnitude $\|\theta_0\|^2$ inherited from whatever initialization or pre-training we choose; (ii) the alignment term C, which rewards or penalizes that initialization depending on how well it already points in the eventual descent direction; and (iii) the learning-rate to batch-size scaled aggregate $(S_1{+}S_2{+}S_3)$, whose geometry tracks how stochastic gradients accumulate, through their individual strength, their coherence inside each batch, and their persistence across time.

A. Practical Perspective

While the decomposition in Eq. 11 is exact, it is not directly tractable in practice for large models. Computing all the trajectory terms requires storing and evaluating inner products between all sample-level gradients across all training steps, which is infeasible at scale. To obtain a practical diagnostic, we assume that gradient dependence is dominated by interactions between consecutive mini-batches. In particular, we approximate cross-step interactions as negligible beyond one iteration, i.e. $G_{B_t}^\top G_{B_u} \approx 0$ for $|t-u| > 1$. With this simplification, $\|\theta_T\|^2$ can be approximated as

$$\|\theta_T\|^2 \approx \|\theta_0\|^2 + C + \underbrace{\eta^2 \sum_{t=0}^{T-1} \|G_{B_t}\|^2}_{S_1+S_2} + \underbrace{2\eta^2 \sum_{t=1}^{T-1} G_{B_{t-1}}^\top G_{B_t}}_{S_3}, \tag{12}$$

The first term corresponds to the accumulated gradient energy and intra-batch alignment $(S_1 + S_2)$, while the second term captures cross-step interference (S_3). To make this interaction interpretable, we rewrite the inner product between consecutive gradients using cosine similarity,

$$\rho_t = \frac{G_{B_{t-1}}^\top G_{B_t}}{\|G_{B_{t-1}}\|\,\|G_{B_t}\|}. \tag{13}$$

Substituting back into Eq. 12 yields the approximation

$$\|\theta_T\|^2 \approx \|\theta_0\|^2 + C + \underbrace{\eta^2 \sum_{t=0}^{T-1} \|G_{B_t}\|^2}_{S_1+S_2} + \underbrace{2\eta^2 \sum_{t=0}^{T-1} \rho_t \,\|G_{B_{t-1}}\|\,\|G_{B_t}\|}_{S_3}. \tag{14}$$

Although tracking consecutive-gradient alignment is not itself new [29], our contribution is to place ρ_t within the signal–interference decomposition and show that it directly governs the cross-step term S_3, thereby linking short-horizon gradient coherence to trajectory stability and loss reduction (Eq. (15)). In particular, $\rho_t > 0$ points to constructive accumulation of learning signal, $\rho_t < 0$ indicates destructive interference, and $\rho_t \approx 0$ indicates random drift. Consistent with this picture, Figure 1 illustrates the trade-off: plain SGD operates in a destructive-interference regime $(S_3/(S_1 + S_2) < 0)$, whereas momentum shifts it toward constructive accumulation $(S_3/(S_1 + S_2) > 0)$, which is associated with lower loss; adding weight decay further improves robustness across a broader range of learning rates.

B. Effects on Loss Dynamics

The proposed framework provides insights into how gradient interactions affect the short-term evolution of the loss function. Consider two consecutive SGD steps $\theta_{t+1} = \theta_t - \eta\, G_{B_t}$ and $\theta_{t+2} = \theta_{t+1} - \eta\, G_{B_{t+1}}$. Using the noise decomposition from Eq. (4), we write $G_{B_t} = g^{(t)} + \xi_t$, where $g^{(t)} := \nabla\mathcal{L}(\theta_t)$ is the true gradient and ξ_t is the zero-mean stochastic noise with $\mathrm{Cov}(\xi_t \mid \theta_t) \approx \frac{1}{m}\Sigma(\theta_t)$. Applying a second-order Taylor expansion of the loss at each step and a first-order expansion of the gradient $g^{(t+1)}$ around θ_t (see Appendix A), we obtain

$$\begin{aligned}\mathcal{L}(\theta_{t+2}) \;\approx\; & \mathcal{L}(\theta_t) \\ & - \eta\Big(\|G_{B_t}\|^2 \;+\; \|G_{B_t}\|\,\|G_{B_{t+1}}\|\,\rho_{t+1}\Big) \\ & + \eta\,\xi_t^\top\big(G_{B_t} + G_{B_{t+1}}\big) \\ & + \eta^2\, G_{B_t}^\top H_t\, G_{B_{t+1}} \\ & + \frac{\eta^2}{2}\Big(G_{B_t}^\top H_t\, G_{B_t} \;+\; G_{B_{t+1}}^\top H_{t+1}\, G_{B_{t+1}}\Big),\end{aligned} \tag{15}$$

where $H_t = \nabla^2\mathcal{L}(\theta_t)$, ρ_{t+1} is the cosine similarity between consecutive mini-batch gradients, and $\xi_t = G_{B_t} - g^{(t)}$ is the sampling noise at step t. A detailed derivation is provided in Appendix A.

Analyzing the two terms proportional to η in Eq. (15), one can see three distinct elements that affect the two-step loss change: (i) per-step gradient energy $\|G_{B_t}\|^2$, which captures the loss decrease from the current mini-batch update alone; (ii) cross-step coherence $\|G_{B_t}\|\,\|G_{B_{t+1}}\|\,\rho_{t+1}$, which determines whether consecutive updates reinforce or partially cancel; and (iii) noise–trajectory coupling $\xi_t^\top(G_{B_t} + G_{B_{t+1}})$, which measures how stochastic fluctuations project onto the combined update direction. Again, when $\rho_{t+1} > 0$, consecutive updates point in similar directions, so the second step builds on the first and amplifies the loss decrease. When $\rho_{t+1} < 0$, the second update opposes the first, so part of the progress made at step t is canceled at step $t{+}1$. This may produce oscillatory behavior and dampen loss reduction across the two steps, even when local curvature measures still suggest stable training.

Furthermore, the noise–trajectory term $\eta\,\xi_t^\top\big(G_{B_t} + G_{B_{t+1}}\big)$ reveals an opposing source to loss decrease. To make it explicit, consider its interaction with the current mini-batch gradient: $\xi_t^\top G_{B_t} = \xi_t^\top g^{(t)} + \|\xi_t\|^2$. The second term, $\|\xi_t\|^2$, is always nonnegative and therefore contributes positively to the loss change regardless of the direction of the update. This means that even when consecutive gradients are well aligned $(\rho_t > 0)$ or nearly orthogonal $(\rho_t \approx 0)$, stochastic noise can still slow down convergence. We speculate here that $\|\xi_t\|^2$ is aligned with the implicit-regularization argument of SGD, in which gradient noise can bias optimization away from purely deterministic descent and toward flatter, better-generalizing regions of the loss landscape [23], [26], [30].

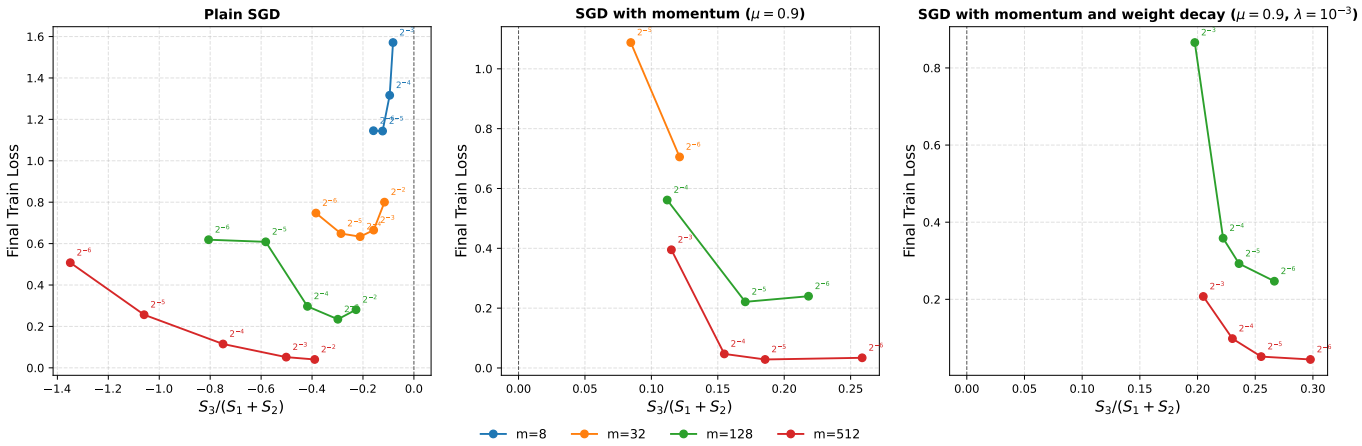


Fig. 1: Final training loss versus the normalized cross-step term $S_3/(S_1 + S_2)$ for VGG in CIFAR-10 with plain SGD (left), SGD with momentum ($\mu = 0.9$) (middle), and SGD with momentum plus weight decay ($\mu = 0.9, \lambda = 10^{-3}$) (right). Colors indicate batch size $m \in \{8, 32, 128, 512\}$; markers are annotated with the learning rate $\{2^{-6}, ..., 2^{-2}\}$. The dashed vertical line denotes the boundary $S_3 = 0$.

The remaining terms proportional to η^2 in Eq. (15) capture curvature effects. In particular, the cross-term $G_{B_t}^\top H_t G_{B_{t+1}}$ shows that curvature influences how successive updates interact by weighting their shared directions through the local Hessian. Because this coupling depends on both batches, it cannot be inferred from $\lambda_{\max}(H_t)$ or $\Sigma(\theta_t)$ alone.

Equation 15 also admits further refinement beyond the present analysis. In particular, it suggests a practical approach for noise diagnostics: if the true gradient $g^{(t)}$ is approximated by a temporal average of mini-batch gradients with memory comparable to the number of batches, e.g. $\hat{g}^{(t)} = (1 - \beta)\,\hat{g}^{(t-1)} + \beta\, G_{B_t}$, where β is a memory coefficient, then the sampling noise can be estimated as $\xi_t \approx G_{B_t} - \hat{g}^{(t)}$. This approximation enables tracking of all first-order (in η) contributions in Eq. (15) along the training trajectory. More broadly, the same expansion could be used to derive step-size conditions that explicitly incorporate cross-step coherence and noise–trajectory coupling, going beyond classical analyses based solely on curvature or worst-case eigenvalues. We leave these extensions for future work, as our focus here is on isolating and characterizing the signal–interference tradeoff.

IV. Mechanisms of Trajectory Stability Under the Signal-Interference Framework

In modern deep learning, training stability is typically achieved through a set of standard heuristics, like momentum [10], [31], weight decay [9], [19], learning-rate schedules [6], [7], adaptive methods such as Adam [17], and more recently gradient orthogonalization [18]. Here we argue that these methods can be viewed as explicit mechanisms for controlling the signal–interference trade-off: they either promote constructive accumulation ($S_1 + S_2$) or mitigate destructive cross-step interference (S_3) by reshaping the magnitude, direction, and temporal structure of the updates.

In this section, we focus on plain SGD, momentum, and weight decay, as they provide the foundational building blocks for adaptive optimization and regularization, which are central to deep learning stability and scale. We first analyze SGD trajectories using the diagnostic in Eq. (14) and identify three regimes: an early constructive phase ($\rho_t > 0$), a destructive phase ($\rho_t < 0$), and a final random drift phase ($\rho_t \approx 0$). We then examine momentum and weight decay, interpreting how each affects the signal–interference trade-off. Next, we run ablations using plain SGD, SGD with momentum, and SGD with weight decay. Finally, we show how batch representativeness explains the coherence and stability of SGD. We follow the benchmark setting used in the Edge-of-Stability literature [13], training VGG [32] on CIFAR-10 [33].

A. Trajectory Regimes in Plain SGD

Here we analyze training dynamics of plain SGD using the cross-step interference proxy introduced in Eq. (14). We trained using plain SGD for a fixed number of iterations ($T = 100{,}000$), varying the batch size over $m \in \{8, 32, 128, 512\}$ while keeping the learning rate fixed at $\eta = 0.01$. Figure 2 shows the smoothed evolution of the training loss, parameter norm, gradient norm, and cosine similarity between successive mini-batch gradients. Because both T and the dataset size N are fixed, larger batch sizes correspond to a larger number of training epochs. Therefore, differences across m reflect both changes in gradient stochasticity and changes in the total amount of data processed during training.

Across all batch sizes, the training trajectories follow a consistent three-phase pattern. In the initial phase, successive gradients are strongly aligned, as indicated by positive cosine similarity, as shown in the bottom-right corner plot of Fig. 2. This alignment is stronger for larger batches and corresponds to rapid loss reduction and steady growth of the parameter norm, reflecting effective accumulation of the learning signal. As training progresses, an oscillatory phase emerges in which cosine similarity becomes negative. This indicates that successive updates partially oppose one another, consistent with overshooting and oscillation around a minimum. In the final

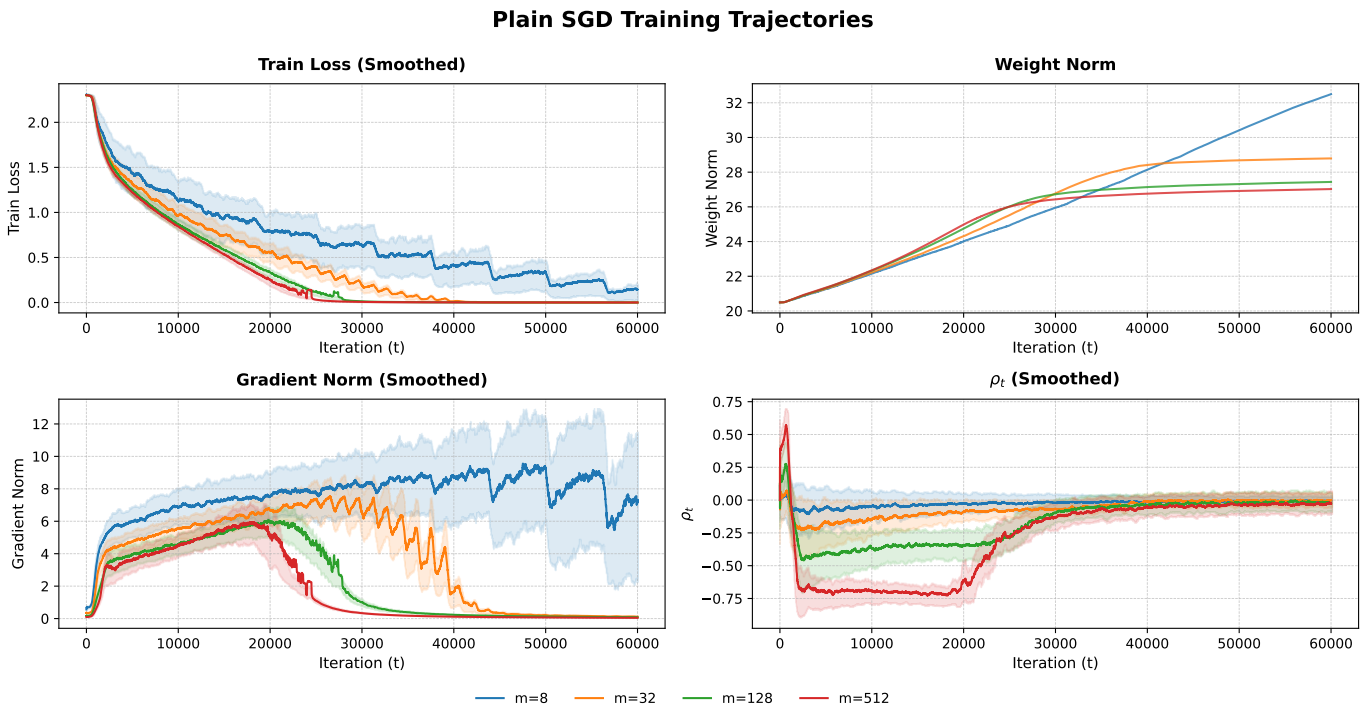


Fig. 2: Smoothed training trajectories for plain SGD with $\eta = 0.01$. The plot grid shows training loss, the model's weight norm, gradient norms, and the cosine similarity between the gradients of sequential batches.

phase, cosine similarity approaches zero, indicating that successive gradients are largely uncorrelated. In this regime, loss improvement saturates, and training becomes noise-dominated: parameter updates behave like a random walk, and further norm growth is driven primarily by stochastic fluctuations rather than a consistent learning signal.

Training dynamics vary systematically with batch size. For larger batches, the three phases are clearly separated, and the optimization trajectory is well structured. Gradient estimates are more accurate, leading to strong early alignment, a clear oscillatory phase near the minimum, and eventual convergence. As the batch size decreases, the magnitude of ρ_t decreases, indicating weaker gradient alignment and larger stochasticity. For the smallest batch size ($m = 8$), training is largely noise-dominated: the loss and gradient norms fluctuate strongly, while the parameter norm continues to grow. In this regime, weight growth is driven primarily by stochastic noise rather than by a consistent learning signal, indicating that noise is being absorbed into the parameters without stable convergence.

B. Momentum as Temporal Averaging

Plain SGD exhibits an oscillatory regime in which consecutive updates partially cancel each other, leading to negative cross-step interference ($S_3 < 0$). Momentum mitigates this behavior by averaging gradients over time, thereby smoothing the trajectory and promoting more consistent update directions. Here, we consider SGD with momentum [10]. Let $\mu \in [0, 1]$ be the momentum coefficient and $p_t \in \mathbb{R}^d$ the velocity, with $p_0 = 0$. The updates are

$$\begin{aligned} p_{t+1} &= \mu p_t + G_{B_t}(\theta_t), \\ \theta_{t+1} &= \theta_t - \eta\, p_{t+1}. \end{aligned} \tag{16}$$

Unrolling the recursion yields $p_{t+1} = \sum_{u=0}^{t} \mu^{t-u} G_{B_u}(\theta_u)$, and telescoping $\theta_{t+1} - \theta_t = -\eta p_{t+1}$ gives

$$\begin{aligned} \theta_T &= \theta_0 - \eta \sum_{t=0}^{T-1} \alpha_t\, G_{B_t}(\theta_t), \\ \alpha_t &:= \sum_{r=0}^{T-1-t} \mu^r = \frac{1-\mu^{T-t}}{1-\mu}. \end{aligned} \tag{17}$$

Expanding the squared norm as in Eq. (10), we obtain the momentum-weighted trade-off decomposition

$$\|\theta_T\|^2 = \|\theta_0\|^2 + C^{(\mu)} + \frac{\eta^2}{m^2}\big(S_1^{(\mu)} + S_2^{(\mu)} + S_3^{(\mu)}\big), \tag{18}$$

where

$$C^{(\mu)} := -2\frac{\eta}{m} \sum_{t=0}^{T-1} \sum_{i=1}^{m} \alpha_t\, \theta_0^\top g_i^{(t)}, \tag{19}$$

$$S_1^{(\mu)} := \sum_{t=0}^{T-1} \alpha_t^2 \sum_{i=1}^{m} \|g_i^{(t)}\|^2, \tag{20}$$

$$S_2^{(\mu)} := \sum_{t=0}^{T-1} \alpha_t^2 \sum_{i=1}^{m} \sum_{\substack{j=1 \\ j\neq i}}^{m} g_i^{(t)\top} g_j^{(t)}, \tag{21}$$

$$S_3^{(\mu)} := 2 \sum_{t=0}^{T-1} \sum_{u=0}^{t-1} \alpha_t \alpha_u \sum_{i=1}^{m} \sum_{j=1}^{m} g_i^{(t)\top} g_j^{(u)}. \tag{22}$$

From Eqs. (17)–(22), momentum can be interpreted as a temporal averaging mechanism. Because each update direction p_{t+1} is an exponentially weighted average of recent mini-batch gradients, momentum smooths high-frequency batch-to-batch variation and tends to make successive updates more coherent. In the signal–interference decomposition, this typically reduces destructive cross-step cancellation and can shift the trajectory toward more constructive accumulation ($S_3 > 0$). At the same time, the coefficients α_t show that earlier gradients have a larger cumulative influence on θ_T, since they continue to affect many future updates through the momentum recursion. This longer memory is beneficial when the descent direction changes slowly over time, but it can be harmful for abrupt direction changes. Such a mismatch can arise from heterogeneous mini-batches, rapid changes in local geometry, or simply an overly large learning rate, which causes the accumulated velocity to overshoot. In that regime, momentum no longer stabilizes the trajectory; instead, it can amplify stale directions and increase oscillatory behavior.

C. Weight Decay as Recency Bias

We next consider SGD with weight decay [9], [19]. Let $\lambda \geq 0$ be the weight–decay coefficient. The update is

$$\begin{aligned}\theta_{t+1} &= \theta_t - \eta\big(G_{B_t}(\theta_t) + \lambda\theta_t\big) \\ &= (1-\eta\lambda)\,\theta_t - \eta\, G_{B_t}(\theta_t).\end{aligned} \tag{23}$$

Unrolling the recursion yields, in the sample-level form,

$$\theta_T = (1-\eta\lambda)^T\theta_0 - \frac{\eta}{m}\sum_{t=0}^{T-1}\sum_{i=1}^{m}(1-\eta\lambda)^{T-1-t}\, g_i^{(t)}. \tag{24}$$

Expanding the squared norm as in Eq. (10), we obtain the weight–decayed trade-off decomposition

$$\begin{aligned}\|\theta_T\|^2 = (1-\eta\lambda)^{2T}\|\theta_0\|^2 + C^{(\mathrm{wd})} \\ + \frac{\eta^2}{m^2}\big(S_1^{(\mathrm{wd})} + S_2^{(\mathrm{wd})} + S_3^{(\mathrm{wd})}\big),\end{aligned} \tag{25}$$

where

$$C^{(\mathrm{wd})} := -2\frac{\eta}{m}\sum_{t=0}^{T-1}\sum_{i=1}^{m}(1-\eta\lambda)^{2T-1-t}\,\theta_0^\top g_i^{(t)}, \tag{26}$$

$$S_1^{(\mathrm{wd})} := \sum_{t=0}^{T-1}(1-\eta\lambda)^{2(T-1-t)}\sum_{i=1}^{m}\|g_i^{(t)}\|^2, \tag{27}$$

$$S_2^{(\mathrm{wd})} := \sum_{t=0}^{T-1}(1-\eta\lambda)^{2(T-1-t)}\sum_{i=1}^{m}\sum_{\substack{j=1\\ j\neq i}}^{m} g_i^{(t)\top} g_j^{(t)}, \tag{28}$$

$$S_3^{(\mathrm{wd})} := 2\sum_{t=0}^{T-1}\sum_{u=0}^{t-1}(1-\eta\lambda)^{2T-2-t-u}\sum_{i=1}^{m}\sum_{j=1}^{m} g_i^{(t)\top} g_j^{(u)}. \tag{29}$$

Weight decay introduces an exponential recency bias along the optimization trajectory. From Eq. (24), the gradient at step t appears in θ_T with coefficient $(1-\eta\lambda)^{T-1-t}$. Hence, a gradient from an earlier step is multiplied by the shrinkage factor $1-\eta\lambda$ more times before training ends, and therefore contributes less to the final parameters. The same pattern carries over to the norm decomposition. In $S_1^{(\mathrm{wd})}$ and $S_2^{(\mathrm{wd})}$, the contribution from step t is weighted by $(1-\eta\lambda)^{2(T-1-t)}$ because the coefficient of that step is squared in $\|\theta_T\|^2$. In the cross-step term $S_3^{(\mathrm{wd})}$, the interaction between steps t and u is weighted by $(1-\eta\lambda)^{2T-2-t-u}$, since it combines one factor $(1-\eta\lambda)^{T-1-t}$ from step t and one factor $(1-\eta\lambda)^{T-1-u}$ from step u. Therefore, both individual contributions and pairwise interactions are increasingly suppressed as they originate farther in the past, while recent updates remain comparatively prominent. When $\eta\lambda \ll 1$, this exponential decay yields an effective memory of approximately $(\eta\lambda)^{-1}$ steps. As a result, outdated gradients have limited influence on the parameter norm, which typically leads to more stable norm growth when the descent direction varies over time.

D. Stability Across Learning Rates and Batch Sizes

We complement the preceding trajectory analysis with an aggregated view of stability across hyperparameters. We sweep the batch size over $m \in \{32, 128, 512\}$ and the learning rate over $\eta \in \{2^{-6}, 2^{-5}, 2^{-4}\}$ for plain SGD, SGD with momentum, and SGD with momentum and weight decay. Figure 3 reports the final training loss, the terminal scaled signal $\frac{\eta^2}{m^2}(S_1+S_2)$ and interference terms $\frac{\eta^2}{m^2}S_3$, and the average gradient cosine similarity $\overline{\rho}_t$. Unlike the trajectories discussed earlier, this sweep compares the behavior across methods at matched (η, m). A central pattern is that larger batches improve all three methods, but they do so through different combinations of signal and interference.

Plain SGD remains in a destructive cross-step regime throughout the sweep. For every $\eta \in \{2^{-6}, 2^{-5}, 2^{-4}\}$ and every $m \in \{32, 128, 512\}$, the average gradient similarity is negative ($\overline{\rho}_t < 0$) and the corresponding cross-step term is also negative ($S_3 < 0$). At a fixed learning rate, decreasing the batch size worsens the loss, increases $S_1 + S_2$, and moves both S_3 and $\overline{\rho}_t$ toward zero. This shows that S_1+S_2 should not be interpreted directly as a more useful signal: it is an update-energy term, and it can grow simply because the gradients become noisier. The sweep suggests that reducing the batch size pushes plain SGD toward a noisy drift regime, whereas increasing it moves SGD toward a more coherent but more strongly interfering regime. Plain SGD thus appears to operate between two unfavorable extremes: random drift at small batch sizes and destructive interference at large batch sizes.

Introducing momentum ($\mu = 0.9$) moves the trajectory into a constructive regime ($\overline{\rho}_t > 0$ and $S_3 > 0$) not seen in plain SGD. For $\eta \in \{2^{-6}, 2^{-5}\}$, the constructive cross-step coincides with lower loss, especially for $m \in \{128, 512\}$. At the more aggressive learning rate $\eta = 2^{-4}$, however, we observe the failure mode of momentum: the loss increases sharply, $S_1 + S_2$ grows dramatically, and S_3 exhibits an unbounded negative excursion. Increasing the batch size mitigates this effect by making consecutive gradients more coherent, which improves the quality of temporal averaging and restores the low-loss constructive regime.

Adding weight decay further improves the momentum regime, mainly in the more aggressive settings. Its clearest benefit appears at $\eta = 2^{-4}$, where it suppresses the growth

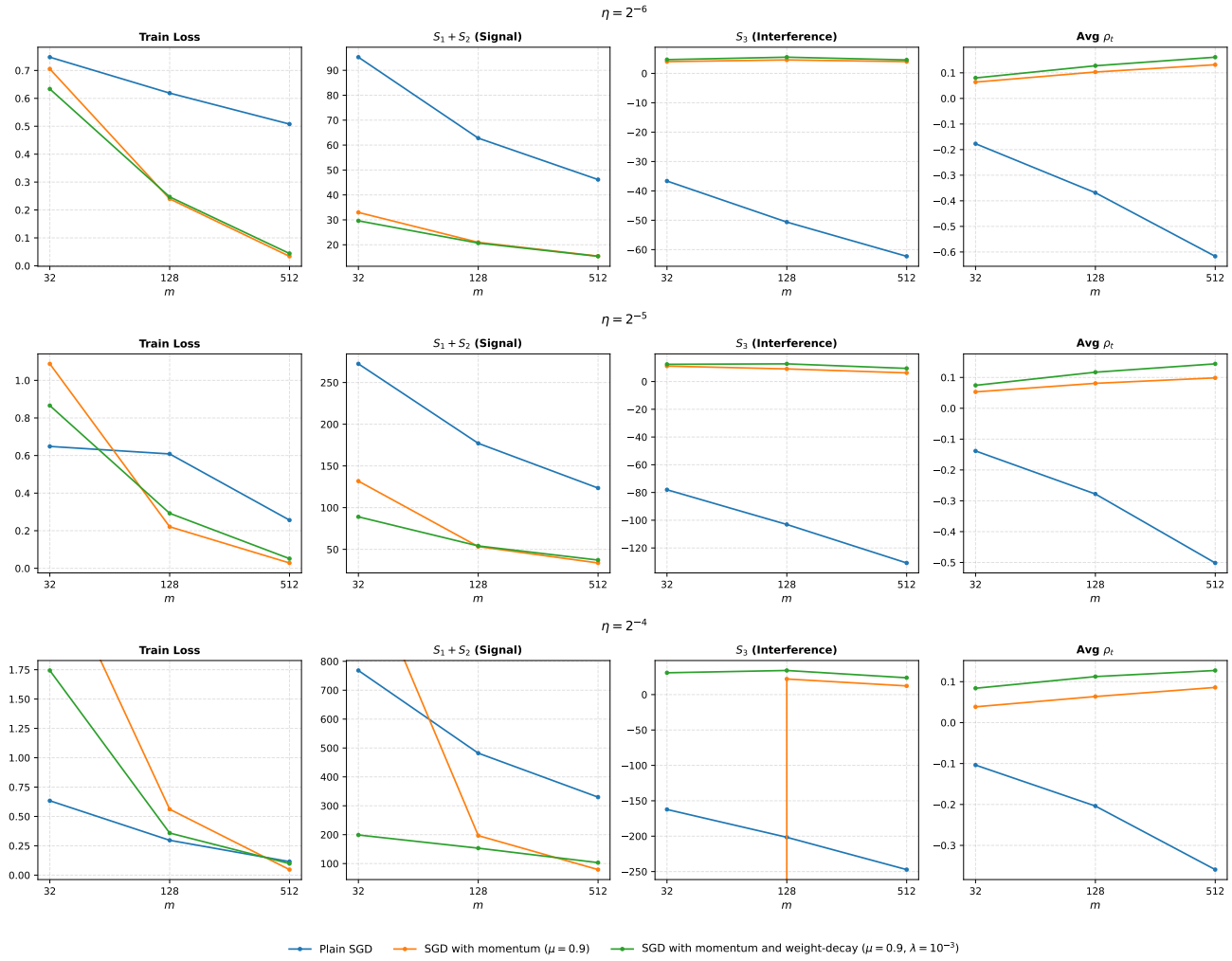


Fig. 3: **Empirical stability ablations for SGD, momentum, and weight decay.** We sweep batch size $m \in \{32, 128, 512\}$ and learning rate $\eta \in \{2^{-6}, 2^{-5}, 2^{-4}\}$. Each point corresponds to a run and reports training loss, scaled signal $S_1 + S_2$, scaled interference S_3, and the average consecutive-gradient cosine similarity $\overline{\rho}_t$.

of $S_1 + S_2$ and avoids the extreme breakdown observed for momentum alone at $m = 32$, preserving positive alignment. In this sense, weight decay makes the momentum regime less sensitive to aggressive hyperparameter choices, consistent with the recency-bias interpretation.

E. Batch Representativeness and Trajectory Coherence

The ablations above suggest that training becomes less stable at small batch sizes because the trajectory becomes increasingly random, as indicated by $\overline{\rho}_t \to 0$ for all cases. Here we argue that this effect is largely driven by limited batch representativeness. Under uniform sampling, each mini-batch $X_t = \{\mathbf{x}_{t,i}\}_{i=1}^m \subset D$ provides only a finite approximation of the training distribution. When m is small, consecutive batches can differ substantially, leading to larger fluctuations in the gradient direction from one step to the next. As m increases, each batch becomes more representative of the dataset, and the resulting updates become more consistent across iterations.

Because our experiments use image data, we quantify batch representativeness directly in input space[1] by measuring the *energy distance* between consecutive mini-batches [34]. Given two successive batches, $X_{t-1} = \{\mathbf{x}_i\}_{i=1}^m$ and $X_t = \{\mathbf{x}_j\}_{j=1}^m$, we estimate this distance as

$$\begin{aligned}\widehat{\mathcal{E}}(X_{t-1}, X_t) = \frac{2}{m^2} \sum_{i,j} \|\mathbf{x}_i - \mathbf{x}_j\| \\ - \frac{1}{m(m-1)} \sum_{i \neq i'} \|\mathbf{x}_i - \mathbf{x}_{i'}\| \\ - \frac{1}{m(m-1)} \sum_{j \neq j'} \|\mathbf{x}_j - \mathbf{x}_{j'}\|.\end{aligned} \tag{30}$$

To effectively control $\widehat{\mathcal{E}}(X_{t-1}, X_t)$, we hold m fixed and break representativeness on purpose, violating the i.i.d. sampling assumption and constructing same-class mini-batches. Our hypothesis is as follows: assuming that batch size improves

[1]For text data, an analogous notion of batch representativeness may be more naturally measured in embedding space, where distances can better reflect semantic similarity than raw token-space comparisons.

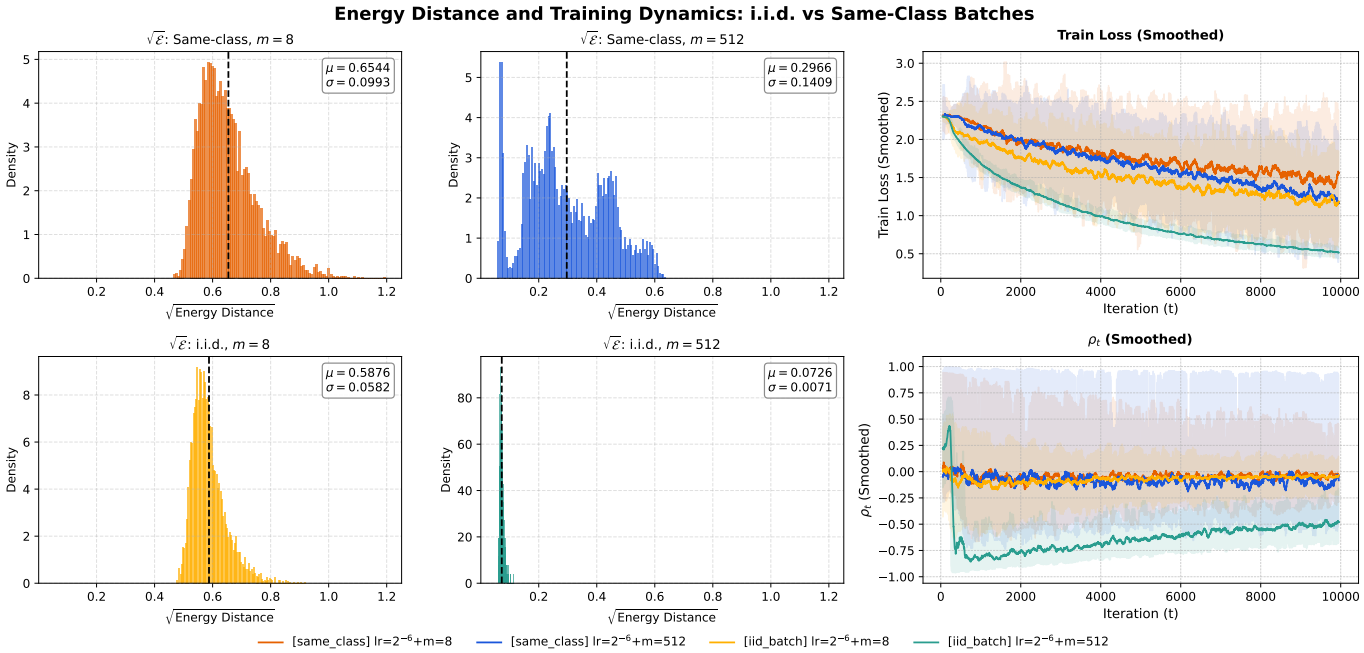


Fig. 4: **Effect of batch construction on consecutive-batch energy distance, and noisy trajectories at large batch sizes.** Histograms comparing same-class and i.i.d. sampling for $m = 8$ and $m = 512$. The horizontal axis shows $\sqrt{\mathcal{E}(X_{t-1}, X_t)}$ (used to expand the scale for readability). Same-class sampling has a minor effect for $m = 8$ (μ: 0.58 $\rightarrow$ 0.65) but a large effect for $m = 512$ (μ: 0.07 $\rightarrow$ 0.29), indicating substantially higher inter-batch discrepancy at large batch sizes. Training loss and ρ_t trajectories: same-class grouping prevents large batches from becoming representative, keeping consecutive updates weakly aligned (small average ρ_t) and yielding random-walk-like drift even at $m = 512$.

representativeness (i.e., reduces step-to-step energy distance), then breaking representativeness at a large batch size will reproduce small-batch interference dynamics and worse optimization behavior.

Figure 4 shows that the energy distance distribution between consecutive batches decreases with batch size under i.i.d. sampling, while breaking representativeness through same-class batching increases both its mean and variability. This effect is much stronger at $m = 512$ than at $m = 8$, suggesting that very small batches already operate far from the i.i.d. regime. The corresponding learning trajectories support this interpretation: when representativeness is disrupted, even large batches recover the noisy, weakly aligned dynamics usually associated with small-batch training. In particular, at fixed $m = 512$, cross-step interference collapses into random drift, and the updates accumulate more like a random walk than a consistent descent direction. These results suggest that much of the batch-size effect is mediated by representativeness: larger batches help because they make successive updates more statistically similar and therefore more temporally coherent. A natural conjecture is that this issue becomes more severe in the scaling regime: as input spaces grow and training data become more diverse, the representativeness of any fixed-size mini-batch should deteriorate, making cross-step coherence harder to maintain and training correspondingly more unstable.

V. Conclusion

The central claim of this paper is that the stability of SGD is not fully explained by pointwise properties of the loss landscape, such as local curvature. Instead, it also depends on temporal interactions between successive mini-batch updates, an aspect that remains comparatively underexplored despite its influence on the training trajectory. To address this gap, we introduced a signal–interference framework for stochastic optimization. By decomposing parameter growth into gradient signal and cross-step interference, the framework brings to light an important property of training dynamics: whether successive updates accumulate constructively or partially overwrite one another. We further derived a formulation to explain such interaction through consecutive-gradient cosine similarity, and used it to analyze plain SGD, momentum, and weight decay. Our results indicate that both stability and performance of SGD depend heavily on the combination of update magnitude and coherence. Momentum improves performance by making successive updates constructive, but it can become unstable if its velocity overshoots. Weight decay improves robustness by reducing the influence of older, stale directions. Overall, our findings complement existing approaches by introducing a temporal perspective on optimization dynamics, together with a practical diagnostic tool for assessing stability based on the correlation between successive updates. Therefore, these contributions provide a more mechanistic understanding of training evolution, helping to bridge the gap between theory and practice in modern machine learning.

APPENDIX A
TWO-STEP LOSS REDUCTION

We derive Eq. (15) by combining a second-order Taylor expansion of the loss across two consecutive SGD steps with a first-order expansion of the gradient. Consider

$$\theta_{t+1} = \theta_t - \eta\, G_{B_t}, \quad \theta_{t+2} = \theta_{t+1} - \eta\, G_{B_{t+1}},$$

and recall

$$G_{B_t} = g^{(t)} + \xi_t, \quad g^{(t)} := \nabla\mathcal{L}(\theta_t), \quad H_t := \nabla^2\mathcal{L}(\theta_t).$$

A second-order expansion of $\mathcal{L}(\theta_{t+1})$ around θ_t gives

$$\begin{aligned}\mathcal{L}(\theta_{t+1}) \approx \mathcal{L}(\theta_t) \\ - \eta\, g^{(t)\top} G_{B_t} + \frac{\eta^2}{2}\, G_{B_t}^\top H_t\, G_{B_t}.\end{aligned} \tag{31}$$

Similarly, expanding $\mathcal{L}(\theta_{t+2})$ around θ_{t+1} yields

$$\begin{aligned}\mathcal{L}(\theta_{t+2}) \approx \mathcal{L}(\theta_{t+1}) \\ - \eta\, g^{(t+1)\top} G_{B_{t+1}} + \frac{\eta^2}{2}\, G_{B_{t+1}}^\top H_{t+1}\, G_{B_{t+1}}.\end{aligned} \tag{32}$$

Next, using a first-order expansion of the gradient around θ_t,

$$g^{(t+1)} \approx g^{(t)} + H_t(\theta_{t+1} - \theta_t) = g^{(t)} - \eta\, H_t\, G_{B_t},$$

so that

$$-\eta\, g^{(t+1)\top} G_{B_{t+1}} \approx -\eta\, g^{(t)\top} G_{B_{t+1}} + \eta^2\, G_{B_t}^\top H_t\, G_{B_{t+1}}. \tag{33}$$

Substituting Eqs. (31) and (33) into Eq. (32), we obtain

$$\begin{aligned}\mathcal{L}(\theta_{t+2}) \approx \mathcal{L}(\theta_t) - \eta\, g^{(t)\top} G_{B_t} - \eta\, g^{(t)\top} G_{B_{t+1}} \\ + \eta^2\, G_{B_t}^\top H_t\, G_{B_{t+1}} + \frac{\eta^2}{2}\, G_{B_t}^\top H_t\, G_{B_t} \\ + \frac{\eta^2}{2}\, G_{B_{t+1}}^\top H_{t+1}\, G_{B_{t+1}}.\end{aligned} \tag{34}$$

Using $g^{(t)} = G_{B_t} - \xi_t$, the two first-order terms become

$$g^{(t)\top} G_{B_t} = \|G_{B_t}\|^2 - \xi_t^\top G_{B_t},$$

$$g^{(t)\top} G_{B_{t+1}} = G_{B_t}^\top G_{B_{t+1}} - \xi_t^\top G_{B_{t+1}}.$$

Thus,

$$\begin{aligned}\mathcal{L}(\theta_{t+2}) \approx \mathcal{L}(\theta_t) - \eta\Big(\|G_{B_t}\|^2 + G_{B_t}^\top G_{B_{t+1}}\Big) \\ + \eta\, \xi_t^\top\big(G_{B_t} + G_{B_{t+1}}\big) + \eta^2\, G_{B_t}^\top H_t\, G_{B_{t+1}} \\ + \frac{\eta^2}{2}\Big(G_{B_t}^\top H_t\, G_{B_t} + G_{B_{t+1}}^\top H_{t+1}\, G_{B_{t+1}}\Big).\end{aligned} \tag{35}$$

Finally, by the definition of the cosine similarity between consecutive mini-batch gradients,

$$\rho_{t+1} = \frac{G_{B_t}^\top G_{B_{t+1}}}{\|G_{B_t}\|\,\|G_{B_{t+1}}\|},$$

$$G_{B_t}^\top G_{B_{t+1}} = \|G_{B_t}\|\,\|G_{B_{t+1}}\|\,\rho_{t+1}.$$

Substituting this into Eq. (35) yields Eq. (15).

REFERENCES

[1] J. Hestness, S. Narang, N. Ardalani, G. Diamos, H. Jun, H. Kianinejad, M. Patwary, Y. Yang, and Y. Zhou, "Deep learning scaling is predictable, empirically," *arXiv preprint arXiv:1712.00409*, 2017.

[2] J. Kaplan, S. McCandlish, T. Henighan *et al.*, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[3] T. Henighan *et al.*, "Scaling laws for autoregressive generative modeling," *arXiv preprint arXiv:2010.14701*, 2020.

[4] J. Hoffmann, S. Borgeaud, A. Mensch, E. Cassirer, T. Jones, T. Dasilva, K. Tunyasuvunakool, C. Humphreys, E. Rutherford, E. Buchatskaya *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[5] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, large minibatch sgd: Training imagenet in 1 hour," *arXiv preprint arXiv:1706.02677*, 2017.

[6] L. N. Smith, "Cyclical learning rates for training neural networks," in *2017 IEEE winter conference on applications of computer vision (WACV)*. IEEE, 2017, pp. 464–472.

[7] I. Loshchilov and F. Hutter, "Sgdr: Stochastic gradient descent with warm restarts," *arXiv preprint arXiv:1608.03983*, 2016.

[8] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International conference on machine learning*. pmlr, 2015, pp. 448–456.

[9] A. Krogh and J. Hertz, "A simple weight decay can improve generalization," *Advances in neural information processing systems*, vol. 4, 1991.

[10] B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Computational Mathematics and Mathematical Physics*, vol. 4, pp. 1–17, 1964.

[11] D. P. Bertsekas, "Nonlinear programming," *Journal of the Operational Research Society*, vol. 48, no. 3, pp. 334–334, 1997.

[12] S. Jastrzebski, M. Daniluk, and K. Cho, "The break-even point on optimization trajectories of deep neural networks," in *International Conference on Learning Representations (ICLR)*, 2020.

[13] J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar, "Gradient descent on neural networks typically occurs at the edge of stability," in *International Conference on Learning Representations (ICLR)*, 2021.

[14] S. Arora, Z. Li, and A. Panigrahi, "Understanding gradient descent on the edge of stability in deep learning," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 948–1024.

[15] A. Damian, E. Nichani, and J. D. Lee, "Self-stabilization: The implicit bias of gradient descent at the edge of stability," in *International Conference on Learning Representations (ICLR)*, 2023.

[16] R. M. French, "Catastrophic forgetting in connectionist networks," *Trends in cognitive sciences*, vol. 3, no. 4, pp. 128–135, 1999.

[17] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[18] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein, "Muon: An optimizer for hidden layers in neural networks," 2024. [Online]. Available: https://kellerjordan.github.io/posts/muon/

[19] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2017.

[20] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur, "Sharpness-aware minimization for efficiently improving generalization," in *International Conference on Learning Representations (ICLR)*, 2021.

[21] L. Sagun, U. Evci, V. U. Guney, Y. Dauphin, and L. Bottou, "Empirical analysis of the hessian of over-parametrized neural networks," 2017.

[22] B. Ghorbani, S. Krishnan, and Y. Xiao, "An investigation into neural net optimization via hessian eigenvalue density," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 2232–2241.

[23] S. Mandt, M. D. Hoffman, and D. M. Blei, "Stochastic gradient descent as approximate bayesian inference," *Journal of Machine Learning Research*, vol. 18, pp. 1–35, 2017. [Online]. Available: https://www.jmlr.org/papers/v18/17-214.html

[24] W. Hu, C. J. Li, L. Li, and J.-G. Liu, "On the diffusion approximation of nonconvex stochastic gradient descent," *Annals of Mathematical Sciences and Applications*, vol. 4, no. 1, pp. 3–32, 2019.

[25] S. Mandt, M. D. Hoffman, and D. M. Blei, "Stochastic gradient descent as approximate bayesian inference," *Journal of Machine Learning Research*, vol. 18, no. 1, pp. 4873–4907, 2017.

[26] S. Jastrzebski, Z. Kenton, D. Arpit, N. Ballas, A. Fischer, A. Storkey, and Y. Bengio, "Three factors influencing minima in sgd," in *International Conference on Learning Representations (ICLR)*, 2018.

[27] K. Lyu and J. Li, "Gradient descent maximizes the margin of homogeneous neural networks," in *International Conference on Learning Representations (ICLR)*, 2020.
[28] Z. Ji and M. Telgarsky, "Gradient descent aligns the layers of deep linear networks," in *JMLR*, vol. 21, no. 1, 2020, pp. 1–34.
[29] C. Xing, D. Arpit, C. Tsirigotis, and Y. Bengio, "A walk with sgd," *arXiv preprint arXiv:1802.08770*, 2018.
[30] S. L. Smith and Q. V. Le, "A bayesian perspective on generalization and stochastic gradient descent," *ICLR*, 2018.
[31] L. N. Smith, "A disciplined approach to neural network hyper-parameters: Part 1–learning rate, batch size, momentum, and weight decay," *arXiv preprint arXiv:1803.09820*, 2018.
[32] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *International Conference on Learning Representations (ICLR)*, 2015, arXiv:1409.1556.
[33] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.
[34] G. J. Székely and M. L. Rizzo, "Energy statistics: A class of statistics based on distances," *Journal of statistical planning and inference*, vol. 143, no. 8, pp. 1249–1272, 2013.